Project: Orientation estimation using smartphone sensors

Mubina Crnic (mubina)
Osama Al Sheikh Ali (osamas)
Group 37

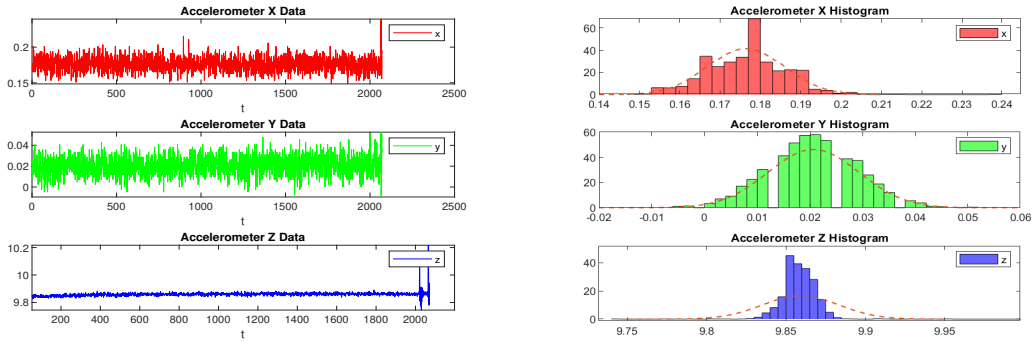
May 24, 2024

Task 1

Some general reasons for using gyroscope data is that they represent angular velocity, a vital ingredient in estimating the orientation of the estimation. This is because it gives information on the rate of rotation. However general problems with gyroscope is that they have have some bias and drift over time which is caused by the accumulation of errors over time. Using gyroscope measurements as inputs $u_k = \omega_k$ makes the state vector simpler and reduces computation costs. This means that our state vector has less parameters which also means that the A matrix is smaller. For this to be possible then the gyro measurement should not be biased and should be rather accurate and in our case (smartphones) the gyroscope from the phone is in fact accurate. Observe that this is only possible for small amount of data but the larger the data the higher risk of accumulative errors.

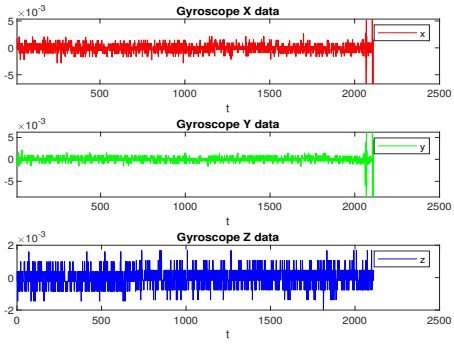
However, a case where it would be better to include angular velocities in the state vector is if the gyro readings have bad accuracy and in doing so the measurement noise from gyro readings can be modeled and accounted for in the motion model. This means that we we model the the gyro measurements as probabilistic measurements instead of being 100% deterministic .

Task 2

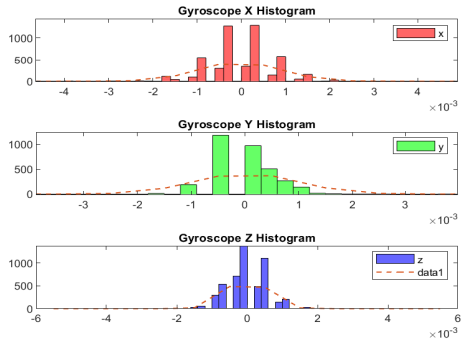


(a) Accelerometer Data *(b) Accelerometer Histogram*

Figure 1: Accelerometer

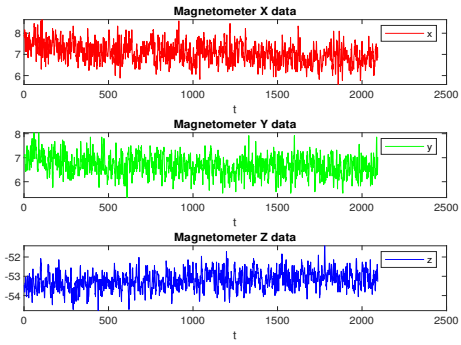


(a) Gyroscope Data

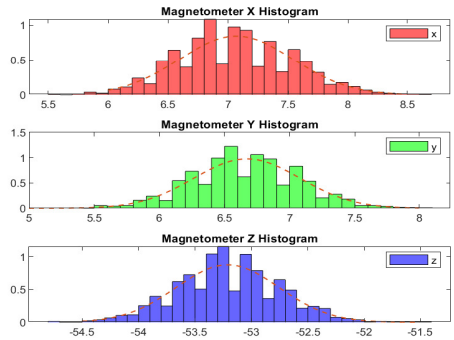


(b) Gyroscope Histogram

Figure 2: Gyroscope



(a) Magnetometer Data



(b) Magnetometer Histogram

Figure 3: Magnetometer

Mean	x	y	z
Acc	1.7640e-01	2.0641e-02	9.8593e+00
Gyr	-4.3752e-05	1.9904e-05	-9.4452e-06
Mag	7.0680e+00	6.6721e+00	-5.3211e+01

Table 1: Mean of the acceleration vector, angular velocity vector, and magnetic field when the phone is placed flat on table

Var	x	y	z
Acc	9.1093e-05	7.4411e-05	5.7983e-04
Gyr	8.1544e-07	9.0072e-07	3.1042e-07
Mag	2.2340e-01	1.6720e-01	2.0757e-01

Table 2: Covariance of the acceleration vector, angular velocity vector, and magnetic field when the phone is placed flat on table

Conclusions: For this task, we used a Samsung s9+ Android phone placed flat on a table. The readings from the various sensors (accelerometer, gyroscope, magnetometer) appeared to follow a normal Gaussian distribution. This was evident from the histogram plots, where the data was successfully approximated by a Gaussian distribution based on calculated means and covariance for each type of sensor reading.

- The mean readings from the accelerometer in the x and y directions are quite small due to various types of vibrations and imperfections in the table. However, the mean reading in the z direction is 9.8593, which reflects the effect of gravity.
- Since the mobile phone was lying flat, the gyroscope readings were expected to be 0 with some normally distributed noise variations. This expectation was confirmed by the plots.
- The readings from the magnetometer were notably noisy and did not follow a Gaussian distribution. However, after multiple attempts, we managed to obtain relatively consistent results. We moved the phone to various locations on the table, seeking an area with stable magnetic fields, minimizing the influence of other electronic devices or metals that could affect the magnetometer readings.

Task 3

Continuous time model:

$$\dot{q}(t) = \underbrace{\frac{1}{2}S(\omega_{k-1} + v_{k-1})}_{A}\, q(t) \qquad \textit{for } t \in [t_{k-1}, t_k) \tag{1}$$

$$\dot{q}(t) = \underbrace{\left(\frac{1}{2}S(\omega_{k-1}) + \frac{1}{2}S(v_{k-1})\right)}_{A} q(t) \tag{2}$$

Using the relation:

$$\exp A \approx I + A$$

the discretized model of the form:

$$q_k = F(\omega_{k-1})q_{k-1} + G(\hat{q}_{k-1})v_{k-1} \tag{3}$$

where $q_k = q(t_k)$ and $q_{k-1} = q(t_{k-1})$ can be derived.
With $A_d = \exp(AT)$ the expression can be derived as:

$$A_d = I + AT = I + \frac{T}{2}S(\omega_{k-1}) + \frac{T}{2}S(v_{k-1}) \tag{4}$$

$$q_k = \underbrace{\left(I + \frac{T}{2}S(\omega_{k-1}) + \frac{T}{2}S(v_{k-1})\right)}_{A_d} q_{k-1} \tag{5}$$

Using:

$$\frac{1}{2}S(\omega)q = \frac{1}{2}\bar{S}(q)\omega$$

and by approximating the term $G(q_{k-1})v_{k-1}$:

$$q_k = \underbrace{\left(I + \frac{T}{2}S(\omega_{k-1})\right)}_{F(\omega_{k-1})} q_{k-1} + \underbrace{\left(\frac{T}{2}\bar{S}(\hat{q}_{k-1})\right)}_{G(\hat{q}_{k-1})} v_{k-1} \tag{5}$$

The EKF linearizes around an estimate of the state at each time step. Given that $\hat{q}_{k-1}$ is our best estimate of q_{k-1} at time t_{k-1}, we use this estimate to approximate nonlinear terms. By using $\hat{q}_{k-1}$ in place of q_{k-1}, we make the computation feasible and maintain the consistency of the linearization process.

This approach aligns with the EKF procedure and is reasonable because the EKF updates and corrects the state estimates iteratively, ensuring that the estimates remain close to the true states as the filter progresses.

Task 4

```
function [x, P] =  tu_qw(x, P, omega, T, Rw)

    %qkplus1 = F * q  + G * vk
    G = T/2*Sq(x);
    F = eye(size(Somega(omega),1)) + T/2 * Somega(omega);

    x = F * x;

    %Ax --> Cov(Ax) = A* Cov(x)* A'
    P = F*P*F' + G*Rw*G';
end
```

Task 5

The modifications made in filterTemplate are shown in the code below:

```
        gyr = data(1, 5:7)';
        if ~any(isnan(gyr))  % Gyro measurements are
            available.
        [x, P] = tu_qw(x, P, gyr, t-t0-meas.t(end), Rw); %t-
            meas.t(end)?
         [x, P] = mu_normalizeQ(x, P);
        else
        P = P+0.01*eye(4); %Random walk
        %   % Do something
        end
```

The pictures in Figure 4 were taken when the phone was laying on the table. Subfigure (a) depicts the situation with an 'incorrect' prior, where the phone is tilted. Subfigure (b) demonstrates the condition after shaking the phone and then placing it back on the table.

Starting the simulation with an incorrect prior (the phone tilted) causes our results to deviate from Google's, showing an incorrect orientation of the phone. Since we in this case don't use a magnetometer we will not know the orientation of the

phone in the beginning.

It is important to note that, in our results, the phone's orientation shows a correct rotation if the reference is the phone lying flat on the table. Our reference being the phone on the table correctly indicates the movement. However, since our reference in this scenario is not the phone lying flat, it results in an incorrect orientation.

When the phone is shaken, we see that it leads to an incorrect orientation. This is because the vibrations from the shaking introduce irreparable errors in the gyroscope.

(a) Incorrect Prior *(b) Shake*

Figure 4: Orientations

Task 6

$$y_k^a = Q^T(q_k)(g_0 + f_k^a) + e_k^a \tag{6}$$

Since:

$$f_k^a = 0$$

the expression evaluates to:

$$y_k^a = Q^T(q_k)g_0 + e_k^a \tag{7}$$

which in general form is:

$$y_k^a = h(q_k) + r_k \tag{8}$$

and imply that:

$$h(q_k) = Q^T(q_k)g_0 \tag{9}$$

$$r_k = e_k^a \tag{10}$$

The following necessary equations are from the lecture notes:

$$\hat{q}_{k|k} = \hat{q}_{k|k-1} + K_k(y_k - h(\hat{q}_{k|k-1})) \tag{11}$$

$$P_{k|k} = P_{k|k-1} + K_k S_k K_k^T \tag{12}$$

$$S_k = h'(\hat{q}_{k|k-1})P_{k|k-1}h'(\hat{q}_{k|k-1})^T + R_k \tag{13}$$

$$K_k = P_{k|k-1}h'(\hat{q}_{k|k-1})^T S_k^{-1} \tag{14}$$

The function in Matlab:

```
function [x, P] = mu_g(x, P, yacc, Ra, g0)
fa = zeros(size(g0,1),1);

h = Qq(x)'*(g0+fa); %Measurement model matrices h

[Q0, Q1, Q2, Q3] = dQqdq(x);
H = [Q0'*g0, Q1'*g0, Q2'*g0, Q3'*g0]; %derivative H

S_k = H*P*H'+Ra; %Innovation S

K_k = P*H'*inv(S_k); %Kalman gain K

%Updating estimate
x = x+K_k*(yacc-h);

%Posterior covariance
P = P-K_k*S_k*K_k';
end
```

Task 7

```
acc = data(1, 2:4)';
if ~any(isnan(acc))  % Acc measurements are available.
  % Do something
      [x, P] = mu_g(x, P, acc, Ra, g0);
      [x, P] = mu_normalizeQ(x, P);
end
```

After including the accelerometer update in our EKF filter the estimation of the orientation yielded better and more accurate results in comparison to before. We are able to estimate the rotation in roll and pitch (x and y) quite well in world frame. This is because incorporating the accelerometer data accounts for the drift caused by the gyroscope. However, for the rotation around z (yaw rotations) it will become wrong with time (more drift). This is quite evident after we do the shake test on the table because from the question, the filter assumes that f is 0 (no force except the gravitational force) but when we actually do fast shakes then f is not really 0 but the filter will not be able to account for it since it assumes that f is 0. The filter will account for the heavy change in the measurements values as if the phone has changed its orientation (which isn't necessary the same how the real orientation of the phone) by adding rotational movements in all directions to our estimation. For it work, there must be some filtering of the data for when the values of the accelerometer is super large due to changes in the force.

Task 8

For the outlier detection it was solved by only considering accelerometer data which are within an interval close to the standard gravitational acceleration which is 9.82. In this way we will only consider data which are valid (f=0 as our assumption) and not data points which are affected by some forces since they tend to give wrong estimates as seen from the previous question 7. In other words such outliers are being rejected and thrown away and not contributing to our estimation. As far as visualization goes, now when we do the shake test as in the prev question, the simulation (own filter) seems to be stabilized and not shaky as in the previous question. Some visualization plots can be seen below.

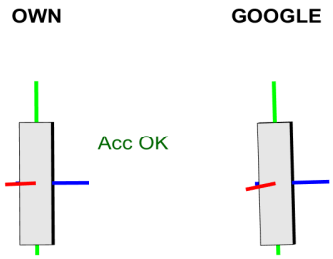


Figure 5: EKF with accelerometer update and outlier rejection algorithm

```
acc = data(1, 2:4)';
if ~any(isnan(acc))  % Acc measurements are available.
  % Do something
  if  norm(acc)>9.81*0.9 && norm(acc)<9.81*1.1
        [x, P] = mu_g(x, P, acc, Ra, g0);
        [x, P] = mu_normalizeQ(x, P);
        ownView.setAccDist(0)
  else
        ownView.setAccDist(1)

  end
end
```

Task 9

$$y_k^m = Q^T(q_k)(m_0 + f_k^m) + e_k^m \tag{15}$$

Since:

$$f_k^m = 0$$

the expression evaluates to:

$$y_k^m = Q^T(q_k)m_0 + e_k^m \tag{16}$$

which in general form is:

$$y_k^m = h(q_k) + r_k \tag{17}$$

and imply that:

$$h(q_k) = Q^T(q_k)m_0 \tag{18}$$

$$r_k = e_k^m \tag{19}$$

The following necessary equations are from the lecture notes:

$$\hat{q}_{k|k} = \hat{q}_{k|k-1} + K_k(y_k - h(\hat{q}_{k|k-1})) \tag{20}$$

$$P_{k|k} = P_{k|k-1} + K_k S_k K_k^T \tag{21}$$

$$S_k = h'(\hat{q}_{k|k-1})P_{k|k-1}h'(\hat{q}_{k|k-1})^T + R_k \tag{22}$$

$$K_k = P_{k|k-1}h'(\hat{q}_{k|k-1})^T S_k^{-1} \tag{23}$$

The function in Matlab:

```
function [x, P] = mu_m(x, P, mag, m0,Rm)


fm = zeros(size(m0,1),1);

h = Qq(x)'*(m0+fm); %Measurement model matrices h

[Q0, Q1, Q2, Q3] = dQqdq(x);
H = [Q0'*m0, Q1'*m0, Q2'*m0, Q3'*m0]; %derivative H

S_k = H*P*H'+Rm; %Innovation S

K_k = P*H'*inv(S_k); %Kalman gain K

x = x+K_k*(mag-h); %Updating estimate

P = P-K_k*S_k*K_k'; %Posterior covariance
end
```

Task 10

By integrating the magnetometer data with the accelerometer and gyroscope data, the sensor fusion algorithm can filter out noise and handle transient disturbances more effectively. This results in a robust and accurate estimation of orientation that is less susceptible to errors from any single sensor. The magnetometer provides an absolute reference for heading (yaw), which helps correct the drift that accumulates in the gyroscope and accelerometer over time. The errors will lead to deviation in the estimation from the googles orientation.
More in details is that a magnetometer provides a fixed point of reference by measuring the Earth's magnetic field.

1. Magnetic North Detection: This measurement allows it to determine the direction of magnetic north. Since the Earth's magnetic field is relatively stable and consistent, the magnetometer can use it as a reliable reference for orientation. This magnetometer measures the earths magnetic filed along (X,Y,Z):

$$m_0 = \begin{pmatrix} 0 \\ \sqrt{m_x^2 + m_y^2} \\ m_z \end{pmatrix}$$

2. Heading Calculation: Using the magnetic north direction, the device can calculate its heading (yaw). This is done by **Angle Calculation:** Using the X and Y components of the magnetic field, the magnetometer calculates the angle between the device's current direction and magnetic north. This is done using trigonometric functions. For example, the arctangent function (`atan2`) is used to compute the heading angle:

$$\text{heading} = \text{atan2}(Y, X) \tag{24}$$

   where X and Y are the horizontal components of the magnetic field.

3. Drift Correction (IMPORTANT STEP): Over time, gyroscopes and accelerometers can accumulate small errors (drift) due to noise and other factors. By comparing their measurements with the stable reference provided by the magnetometer, the system can correct these errors. Thus, the magnetometer provides a consistent reference point, helping to maintain accurate orientation information over time.

The second part of this task has got the same structure as question task 7 since we assume that f is 0 meaning that there are no other effects from the magnetic field however in reality this was not always the truth since we got different magnetic field disturbances from other metals and electronic devices close to the phone which introduced outliers. So in other words the magnetic force f in real life is not 0 but the filter considers these changes as different kind of rotations even though the phone is steady at that position. This will of course lead to weaker performance and our estimations are not ideal. However, when f=0 in other words there are no disturbances from other magnetic fields the magnetometer improves the results (since the heading yaw can be estimated correctly). Finally it can be observed that our own filters is quite similar to the googles filter as expected (when not introduced with magnetic field affects). From the following graph it can be seen that when there is a magnetic field distrubance the performance is not ideal.

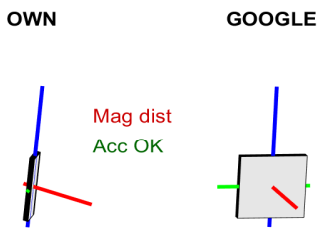


Figure 6: When introducing a magnetic disturbance, own does not match google (correct orientation)

Task 11

To implement outlier detection just like in task 8 but for magnetometer data, we should estimate the strength of the magnetic field and reject measurements when the field strength differs too much from what we expect. To do this we use a simple filter that estimates the magnitude, an AR(1)-filter of the type:

$$\hat{L}_k = (1-\alpha)\hat{L}_{k-1} + \alpha||m_k|| \tag{25}$$

To create the outlier detection and rejection we discarded all values of L outside the interval of $35 < \mathrm{L} < 75$. These values were determined iteratively by checking

how close does the phone has to be close the closest electronic device and these values result in a reasonable distance. The chosen alpha value was 0.02

```
        mag = data(1, 8:10)';
        mag_discard = false;
        if ~any(isnan(mag))  % Mag measurements are available.
          %Do something
          L=(1-alpha)*L+alpha*norm(mag);
          if (L>35 && L<75)
              ownView.setMagDist(0)
          else
              mag_discard = true;
              ownView.setMagDist(1)
          end
        end

      if  magCali_run  && ~mag_discard % run if mag is calibrated and has valid values
              [x, P] = mu_m(x, P, mag, m0, Rm);
              [x, P] = mu_normalizeQ(x, P);
      end
```

By having the outlier detection mechanism, we will only consider magnetometer data which are considered to be stable and have a magnetic field disturbances free area, which is makes relatively possible to assume that the valid measurements from the earth magnetic field and thus we will be able to estimate the yaw angle that is discussed better. As a result we will have better performance of the filter since outliers are being discarded from the estimation. Finally, the following plot we can see that even when we have an outlier our estimation is still being pretty close to the Google one.

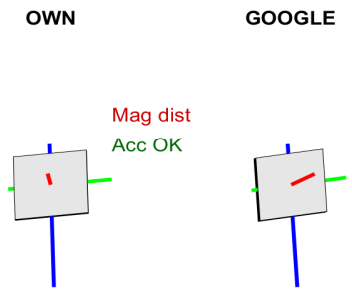


Figure 7: With implementation of an outlier rejection for magnetic disturbances

From the following plot we can see that our estimation is not so bad in comparison to the Googles estimation. It is still obvious that the Googles filter has got better performance.

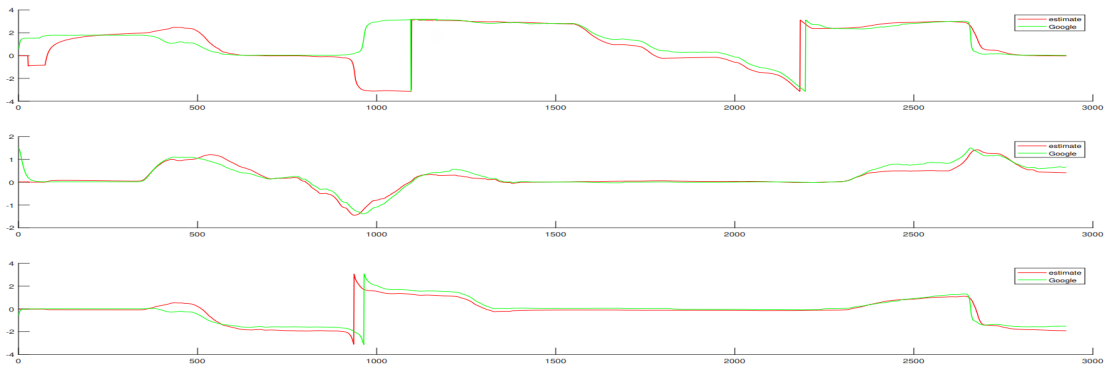


Figure 8: Euler angles of our orientation filter and the built in filter in the phone.

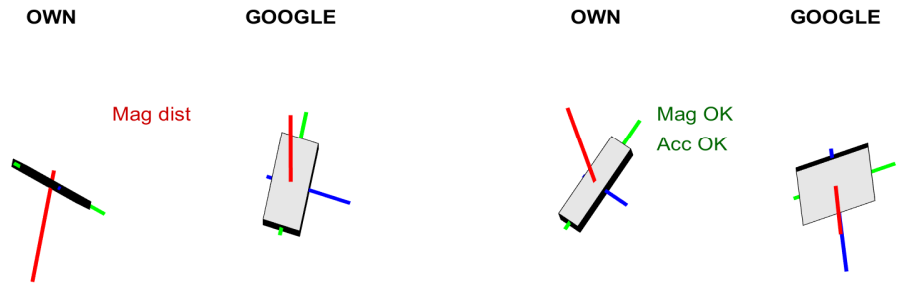
Task 12

For this task different tests were conducted as to test the performance of the filter. First test is to only have the accelerometer and the magnetometer but no gyroscope data. What we observed is that the filter works but is quite slow to follow the rotational motion. Here basically the accelerometer gives the acceleration while the magnetometer is used to determine the orientation of the mobile using the magnetic field of the Earth. Very tiny movements in the mobile resulted in large change because our estimation became quite aggressive. Since the movements were small and quick and the gyroscope in its nature is great for short movements, the

filter could not estimate the googles result correctly.

Moving on to the next test which was that we have all sensors except that accelerometer. Our observation is that if there any movement on our mobile our filter will just fly away and start rotating freely as if it was floating through space. This is because without the accelerometer we are not capturing the linear acceleration correctly.

Finally when we had absence of the magnetometer data, the heading was estimated instead using the gyro which will result in a drift in comparision to when we had the magnetometer.



(a) Test without accelerometer *(b) Test without gyroscope*

Figure 9: Different tests